

(PCC) Data Structures and Complexity Theory

Teaching Scheme	Evaluation Scheme
Lectures: 3 Hrs./ Week	Theory: MSE: 30 Marks, TA: 20 Marks, ESE: 50 Marks
Laboratory: 2 Hrs./ Week	Laboratory: CIE: 100 Marks
Self - study: 1 Hr./week	
Course outcomes	
1. Understand the fundamental concepts of various data structures (e.g., arrays, linked lists, stacks, queues, trees, graphs, hash tables). 2. Understand the concepts of linear and non-linear data structure and their representation 3. Demonstrate the ability to write abstract data types and the use of basic data structures in solving problems. 4. Discuss, compare and implement algorithms for various operations in different 5. implementations of stacks and queues, tree, graph, matrices, heap, etc. 6. Analyze the time complexity of different searching, sorting, and traversal algorithms.	
Course content	
Fundamental Concepts	[6 Hrs]
Introduction to Data Structures: Data, Data Object, Data types, Abstract Data Types (ADT), Data structures, Concept of primitive and non-primitive, linear and non-linear data structures. Introduction to Algorithms: Definition and Characteristics of an algorithm, Algorithm Specification, Performance Analysis- Time and space complexity, Asymptotic notations, Best, Average and worst cases. Introduction to Data representation and files: Text and binary files, use of various libraries for handling files.	
Linear Data Structures and their storage representation	[6 Hrs]
Stack and queue as ADT, Operations on stack and queue. Implementations using arrays and linked lists. Application of stack for expression conversion and evaluation, Recursion. Problems like maze and knight's tours	
Linear Data Structures: Stacks and Queues	[8 Hrs]
Concept of Non-Linear Data Structures, Tree: Basic terminology, representation of trees, Binary Tree: ADT Binary trees, Properties of a Binary Tree and its representations, Binary tree traversals – Inorder, preorder, postorder (recursive and non-recursive) and various operations. Binary Search Tree (BST): Definition, searching a BST, Operations of Insertion and deletion on BST. Heaps: Priority queues, Max and Min Heap, Operations on heap	
Non-Linear Data Structures: Trees	[8 Hrs]
Concept of Non-Linear Data Structures, Tree: Basic terminology, representation of trees, Binary Tree: ADT Binary trees, Properties of a Binary Tree and its representations, Binary tree traversals – In order, preorder, post order (recursive and non-recursive) and various operations. Binary Search Tree (BST): Definition, searching a BST, Operations of Insertion and deletion on BST. Heaps: Priority queues, Max and Min Heap, Operations on heap	
Non-Linear Data Structures: Graphs	[6 Hrs]
Graph ADT, Representation of graphs using adjacency matrix, adjacency list, Elementary graph operations- Breadth First Search (BFS), Depth First Search (DFS), Analysis of BFS and DFS Spanning Trees- Introduction, obtaining minimum cost spanning tree using greedy design strategy of Prim's and Kruskal's algorithms, Single source shortest paths using Dijkstra's algorithm.	
Searching and Sorting:	[6 Hrs]
Linear and binary search algorithms and their analysis. Bubble Sort, Selection sort, Insertion Sort, Quick Sort, Heapsort with time complexity analysis. Hashing: hashing functions, chaining,	

overflow handling with and without chaining, open addressing: linear and quadratic probing.	
Self-study	[12Hrs]
Textbooks	
1. E. Horowitz, S. Sahni, S. Anderson-freed, "Fundamentals of Data Structures in C", Second Edition, University Press, ISBN 978-81-7371-605-8 2. B. Kernighan, D. Ritchie, "The C Programming Language", Prentice Hall of India, Second Edition, ISBN 81-203-0596-5 3. Y. Langsam, M. Augenstein and A. Tannenbaum, "Data Structures using C", Pearson Education Asia, First Edition, 2002, ISBN 978-81-317-0229-1	
Reference Books	
1. Ellis Horowitz, S. Sahni, D. Mehta "Fundamentals of Data Structures in C++", Galgotia Book Source, New Delhi 1995 ISBN 16782928 2. 2. Jean-Paul Tremblay, Paul. G. Sorensan, "An introduction to data structures with Applications", Tata Mc-Graw Hill International Editions, 2nd edition 1984, ISBN-0-07- 462471-7	
Web references	
1. https://onlinecourses.nptel.ac.in/e-learning/preview/noc25_cs81	
Laboratory assignments	
1. Write a program to remove duplicate doubles from an array of doubles. In the program, write a function which accepts an array of doubles and removes the duplicates from the array and has return type void. 2. Compare the time complexity of two sorting algorithms, by following the given steps. Create a set of data files with count of integers varying into thousands and millions. Sort the files using both the algorithms. Plot graph of the time taken by both the programs using tools like gnuplot. Compare the graphs and comment on the time complexity theoretically predicted and practically observed. 3. Write a function which evaluates an infix expression, without converting it to postfix. The input string can have spaces, (,) and precedence of operators should be handled. 4. Implement a queue (that is write queue.c and queue.h only) of characters, such that on an enqueue, the char is added at the end of queue, and on a dequeue the first element is taken out, but the queue uses only a 'head' pointer and not a 'tail' pointer. 5. Write a data type called "Integer". The data type should represent integers of unlimited length. 6. Write a sorting program with the following features: Reads data from a text file and sorts it alphabetically by default. If the file has data in rows and columns (separated by space or tab) then allows sorting on a particular column. Allows any sort using numeric or alphabetical ordering. 7. Write the following functions for a binary search tree implementation: Searches the maximum value in the tree, preorder traversal without using recursion, Search the string in the tree and returns a pointer to the node, print the binary tree so that it looks like a tree.	

8. Write code to list leaf nodes, non-leaf nodes and level of all nodes in a given binary tree.
9. Write a code for level order traversal of a binary tree with and without stack.
10. Start with an empty AVL tree and perform a series of insertions like : December, January, April, March, July, August, October, February, November, May, June. Display the tree.

This list serves as a guideline and is intended to be continuously refined by the instructor.